

Formal Verification of the Lightning Network in TAMARIN

Review layer. Everything in orange is Claude's suggestion or draft; nothing of the authors' text has been reworded or removed. Gray strike-through marks the authors' own words that Claude suggests deleting or replacing (the orange text right after it is the proposed replacement). Orange margin notes ("Claude") are comments and replies to Simon's and Sana's notes. Figures are cropped unchanged from Tarmin_LN_4.pdf.

Abstract—TODO

Suggested abstract: The Lightning Network moves Bitcoin payments off-chain. Its security rests on two mechanisms: revocation, which deters parties from publishing stale channel states, and hash time-locked contracts (HTLCs), which route payments atomically over untrusted intermediaries. We give a machine-checked symbolic model of both layers in TAMARIN: the channel lifecycle with revocation and punishment, composed with multi-hop HTLC forwarding carrying amounts and per-hop fees. Across five theories and 87 lemma checks, under a Dolev-Yao adversary restricted only by explicit liveness assumptions, we prove payment atomicity, intermediary safety, routing causality and value conservation for three-hop payments, and lift causality and value conservation to paths of arbitrary length over an idealised routing layer. Modelling fees quantifies the wormhole attack: the loss is exactly the path fees. We report a modelling pitfall (one-shot on-chain outputs encoded as persistent facts leave every safety lemma intact while allowing unbounded re-sending), fixed by a linear single-spend discipline. Finally, we isolate the claiming obligation T3 as the one assumption intermediary safety rests on: removing it yields a counterexample.

I. INTRODUCTION

Bitcoin [1] is the most important cryptocurrency in terms of market capitalisation. It is, however, very lacking in many usability and scalability aspects: it is costly, has low bandwidth (only a few dozen transactions per second), and high latency. Many protocols strive to alleviate those shortcomings.

a) *Payment Channel Networks:* Notably, payment channels enable a pair of users to quickly and cheaply perform arbitrarily many payments by relying on the blockchain only to resolve disputes. Both agents pool together funds and then iteratively update their balance. The scripts involved and the protocol itself ensure that no honest agent loses money, by relying on a revocation mechanism: any party closing a channel using an outdated favourable state can be punished, consequently losing all funds invested in the channel. However, payment channels are constrained to pairs of users. Two unconnected users still need to establish a channel in order to perform a payment; this is often not cost effective. Thus arose the concept of payment channel networks: one transitively propagates funds across payment channels to perform a payment between two users that are part of the same network but not connected directly. The most popular of such protocols is perhaps the Lightning Network [2], with about 170M\$ locked on the network [3].

Due to the sheer complexity of its specification [4], it suffers from many attacks [5], [6], [7]. And despite promising ongoing efforts [8], [9], [23] it does not yet feature any complete and unbounded verified formalisation. This paper attempts to alleviate that issue.

b) *Protocol Verification:* Designing and analysing cryptographic protocols is a notoriously difficult and error-prone task, as already illustrated by the various attacks and tentative fixes on the Lightning Network. Formal methods provide hope for a more robust verification of protocols, and automation. Protocol verification is further split between two main classes of models: the computational model—inherited from cryptography, where the attackers and the participants are polynomial-time algorithms and results are typically probabilistic—and the symbolic (or Dolev-Yao) model [10]—where the attacker has infinite computational power but is limited to a set term algebra. The symbolic model notably abstracts away from cryptography and focuses on the logic of a protocol, where much of the attacker activity is typically found.

Although some tools exist to help the cryptographer in the computational model, such as CRYPTOVERIF [11], EASY-CRYPT [12], SQUIRREL [13] or CRYPTOVAMPIRE [14], they all are brittle, hard to use and/or very scoped. In contrast, symbolic verification enjoys very successful automated verifiers in the form of PROVERIF [15] and TAMARIN [16]. In particular, TAMARIN is a multiset rewriting-based automated protocol verifier with a large pedigree of widely used protocols verified, such as WIREGUARD [17], TLS 1.3 [18] among many others.

Suggested rewrite: TAMARIN analyses protocols specified as multiset rewriting rules against first-order trace properties, with unbounded verification modulo an equational theory. It has been used on many widely deployed protocols, such as TLS 1.3 [18] and WireGuard [24].

c) *Contributions:* We provide a novel modelisation of the Lightning Network in TAMARIN featuring

- Unbounded channels;
- idealised unbounded routing layer .

channel lifecycle incl. revocation and punishment, composed with multi-hop HTLC forwarding with per-hop fees, verified at fixed path length under full Dolev-Yao, and at unbounded path length over an idealised routing layer with end-to-end value conservation.

Suggested contributions:

- A TAMARIN model of the channel lifecycle (opening, update, revocation, cooperative and unilateral close, pun-

Claude

Cite LeanDY (same group): it excludes fees (Sec. 4.2) and calls liveness out of reach of DY (Sec. 4.2.5) – exactly the gaps this paper fills. Omitting it looks like not knowing it.

Simon

Is this a good intro to TAMARIN?

Simon

There seems to be 50000 models of WireGuard.

Claude

[17] (Lipp et al.) is a CryptoVerif proof, not Tamarin. Tamarin's WireGuard model is Donenfeld-Milner [24]; cite that, or drop WireGuard and keep TLS 1.3 [18], which is Tamarin.

Claude

First thing a

Claude

Write \$170M and date it (as of 2026).

ishment) composed with multi-hop HTLC forwarding carrying per-hop amounts and fees (Sec. II–Sec. V).

- Machine-checked atomicity, intermediary safety, routing causality and value conservation for three-hop payments under a Dolev-Yao adversary restricted only by explicit liveness assumptions, with the claiming obligation T3 shown to be load-bearing (Sec. V).
- A quantitative account of the wormhole attack: the loss is exactly the path fees (Sec. V-C).
- An idealised routing layer lifting causality and value conservation to arbitrary path length (Sec. VI).
- A modelling lesson found in our own model: one-shot on-chain outputs must be linear facts; the persistent encoding left every safety lemma green (Sec. V-0a).

II. PROTOCOL OVERVIEW AND THREAT MODEL

The Lightning Network can be ~~still~~^{split} into two main parts: the ~~Payments~~^{Payment} Channels and the Routing. The first one enables to ~~transfer~~^{transferring} funds quickly ~~en~~^{and} efficiently between two participants with reduced interaction with a blockchain. However, the setup is comparatively expensive.

The routing protocol organises and secures transfers going beyond a single channel, scaling a ~~mean~~^{means} of localized pairwise payments to a whole network.

Each part carries its own cryptographic mechanism. In the payment channel, revocation keys protect against outdated states. In the routing protocol, a payment hash and a deadline control the transfer. They meet only in the commitment transaction, where an in-flight payment is an extra output. It is subject to the same revocation as the rest of the channel. We included it in our model as well. The model has a channel layer and a routing layer. The wormhole attack of sec. V-C touches only the payment hash. So the channel-layer properties all remain intact.

This section gives an overview of these two parts.

A. Payment Channel

The Payment Channel protocol fundamentally enables two agents to commit and revoke a shared blockchain-related “state”. This state typically tracks the balance of some multi-sig [19] common fund, effectively transferring coins between its two participants. The state is represented by valid transactions spending that common fund, and a payment channel ensures these transactions are valid and that no agent can publish an old, favourable state. The latter property comes from careful signing (restricting transactions to agreed-upon shapes) and punishing: a participant may unconditionally retrieve all funds locked in a channel if their counterparty puts an old state on chain.

Concretely, the two parties open a channel. They use an on_chain (on-chain) funding transaction that pays into a 2-of-2 output. After that, they make off-chain payments. They send signed commitment transactions back and forth. They can update the state by exchanging new commitments. When the state is being updated, parties exchange revocation secrets for the previous commitment and revoke the old state. If one

party publishes an old commitment after revocation, the other party may claim all funds. In between state updates, any commitment that has not been revoked yet is considered a valid unilateral close. If it’s being published by either party, there will be no penalty. The punishment will be triggered only when a revoked secret commitment is published.

Channels can be closed either cooperatively, in which both parties agree on the final balance and settle in a single chain transaction, or unilaterally. In a unilateral closing, one party broadcasts an unrevoked commitment. Both parties have to wait for a time lock (CLTV; Common abbreviation for the CheckLockTimeVerify opcode) to expire; this gives the other party time to detect whether the published secret is revoked or not. If a revoked state is published, then the other party can unilaterally punish the cheater and claim the entirety of the channel funds protected by a relative timelock (CSV; Common abbreviation for the CheckSequenceVerify opcode).

Suggested fix: *The publishing party has to wait for a relative timelock (CSV) to expire before spending its own output; this gives the other party time to detect whether the published state was revoked.*

Suggested replacement for the two paragraphs above: *The five phases of a channel — opening, update, revocation, closing and punishment — are detailed in Sec. II-A2.*

In the context of the Lightning Network, the state also tracks in-flight payments which will propagate further than this channel. These are carried by hash time-locked contracts (HTLCs). We discuss HTLCs fully in sec. II-B. The protocol relies on two things: careful handling of revocation keys and careful handling of transaction signatures. Fig. 1 shows the channel lifecycle.

1) *The Transactions:* Although we abstract it away in our model, one should keep in mind the shape of the transaction in the Lightning Network. Fig. 2 gives the shape of the commitment transaction held by Alice in an Alice-Bob Channel. This transaction notably enables unilateral closing. It connects the parts in our rules to the real protocol.

Suggested rewrite of the last two sentences: *Fig. 2 lets the reader locate, in the real transaction, the objects our rules manipulate; only the in-flight outputs and the revocation path on Output [4] appear in the model.*

Outputs [3] and [4] ~~enables~~^{enable} the simple redistribution of the common fund between Alice and Bob, while Outputs [1] and [2] lock the ~~fund of in-flight payment~~^{funds of in-flight payments} specific to the Lightning Network. The model represents only the in-flight outputs and revocation path on Output [4].

The figure also illustrates the punishment mechanism: unlike Bob’s Output [3], Alice needs to wait for a relative timelock (CSV; Common abbreviation for the CheckSequence Verify opcode) before redeeming Output [4]. That gives Bob the opportunity to ~~redeem~~^{redeem} Output [4] using Alice’s revocation key (rk_A). This revocation key is initially hidden from Bob. ~~Alice only releases as a proof~~^{Alice releases it only as proof} she will not use a given state. This mechanism enables declaring states as “old”.

Simon
If I understood correctly it should be CLTV here as this is the timeout for the in flight HTLC which is absolute

Claude
Not here: this is unilateral close. Only the publisher waits, on the RELATIVE to_self_delay (CSV) of its own output; the counterparty’s output is immediate. CLTV (absolute) is the HTLC timeout h_i (Fig. 2, Locking).

Simon
if we keep this paragraph, we need to remove the “CSV...” from below

Claude
Agreed: CSV is spelled out twice (here and in II-A1). Keep the first occurrence only.

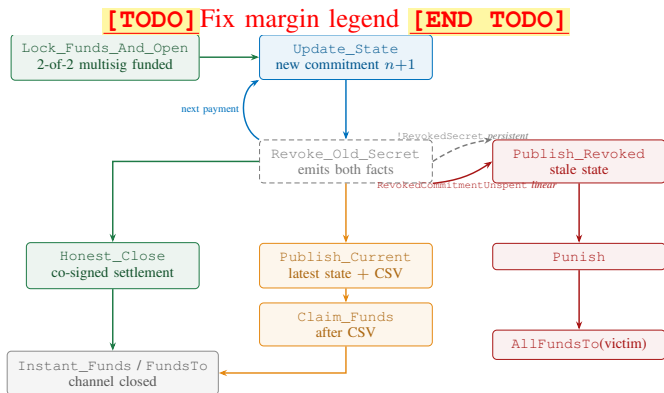
Simon
What do you mean by that?

Claude
Intended meaning: Fig. 2 lets the reader locate the objects our rules manipulate (revocation secret, HTLC outputs, CSV/CLTV) in the real transaction.

Claude

Simon
Do we need the 2 next paragraph. They seem to duplicate content from the ‘subsection’.

Claude
Yes: the next two paragraphs repeat II-A2 (Opening, Update, Revocation, Closing, Punishment). Suggest deleting both and keeping one sentence



Green: open and cooperative close.

Blue: update loop.

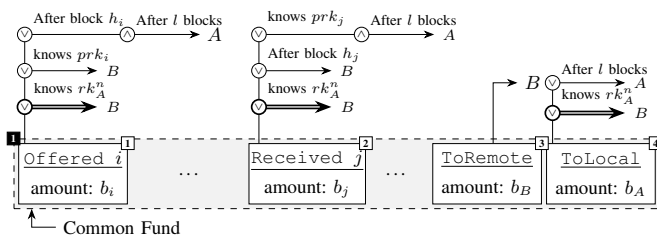
Orange: unilateral close with its CSV delay.

Red: revoked publish and punishment.

Suggested fix for the TODO: move the legend into the caption as one sentence (Green: ...; Blue: ...; Orange: ...; Red: ...); it then cannot overflow the margin.

Revoke_Old_Secret emits *two* facts, and both gate punishment: the persistent !RevokedSecret (dashed, checked but never consumed) and the linear RevokedCommitmentUnspent (solid, consumed once). Gating on the persistent fact alone is the defect of Section V-0a (the “0” appears because that paragraph sits directly under Section V with no subsection; make it a \subsection or cite it by name). Cooperative and unilateral closes end in settlement; punishment terminates in AllFundsTo. Grace-period and pending-publish paths, and the counterparty’s instant output on a unilateral close, are elided.

Fig. 1. Channel lifecycle.



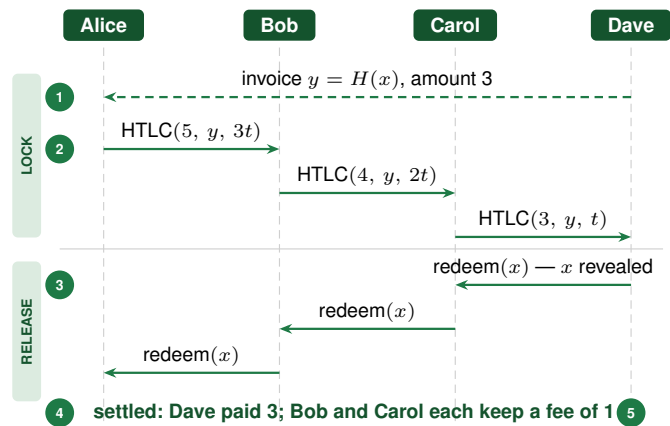
where the CSV is set to l , and the timeout (CLTV) of the k^{th} HTLC is set to height h_k for the Offered and Received outputs; similarly prk_k is the corresponding pre-image of the payment hash; $\Rightarrow$ denotes the punishment path; and **1** refers to an output.

[NOTE] I removed the two staged thing **[END NOTE]**

Source credit needed: the figure is adapted from LeanDY [23] (your own source comment said it was taken as is).

Fig. 2. Model of the Transaction held by A

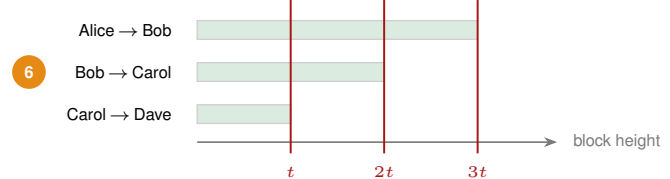
Fig. 2 also describes the local mechanism for in-flight payments. Output **1** (resp. **2**) captures in-flight funds flowing towards Bob (resp. Alice). Besides the punishment mentioned above, these outputs are further governed by payment hashes and a timeout. To fulfill an in-flight transfer, the receiver (Bob in Output **1**’s case) needs the pre-image of the payment hash associated with that transfer. To prevent funds from being locked up, the output effectively reverts to a regular output once a block height is reached (e.g., h_i in **1**’s case). Real-world Lightning Network uses a two staged system to separate the CLTV and CSV. Our



Amounts decrease $5 \rightarrow 4 \rightarrow 3$; each one-unit drop is that intermediary’s fee. Deadlines decrease $3t > 2t > t$, so every forwarder can redeem its incoming HTLC before its outgoing one expires. Circled numbers index the lemma groups of Table II.

(a) Successful three-hop payment

HTLCs are locked as in Fig. 3a; Dave then never releases x , so every HTLC runs to its deadline.



Each window strictly contains the next, so a forwarder always redeems its incoming HTLC before its outgoing one expires.

Capital is locked for the full window and the payment fails, but the nested deadlines $t < 2t < 3t$ mean every honest party is refunded: this is a denial of service, not a loss of funds (Loss_Requires_Inaction).

(b) Griefing

Fig. 3. In-flight Payments

model collapses this complexity into what is shown in Fig. 2.

Suggested text for the modelling section (per Simon’s note): Claiming an in-flight output is a two-stage process: the publisher does not spend Output **1** directly but publishes a second-stage transaction (HTLC-timeout or HTLC-success) whose own output is encumbered by the CSV delay and the revocation path. Our model collapses the two stages into a single transition, because that encumbrance is already represented at the channel level by Publish_Current, Claim_Funds and Publish_Revoked. What we lose is the per-HTLC granularity of the delay, not the delay itself.

2) *The Protocol:* The lifecycle of a channel has five phases: opening, update, revocation, closing, and punishment, which correspond, left to right, to Fig. 1.

a) *Opening:* The opening phase aims to establish a first committed state and the common fund; in that order to prevent griefing attacks [in that order, so that a funder always holds a pre-signed exit]. After an unmodeled negotiation for the inputs of the common fund, Alice and Bob each sample a random revocation key (rk_A^0 and rk_B^0 respectively) and exchange their hashes. Relying on SegWit [20], both parties can compute

Simon
I commented out the rest as I believe it will better fit in the part where the model is explained directly

Simon
align the paragraph names to the figure

Sana
Corrected

Claude
Resolved: order and names now

the common fund TXID and exchange the various signatures needed to spend it: it will be a multisig [19] transaction locked by both ~~party~~**parties**. For instance, Bob signs the various outputs of Fig. 2 (modeled in TAMARIN as a single signature of the whole transaction). Then, the ~~witnesses~~**witnesses** for the common fund are exchanged and it is published. Once on chain, the ~~channel~~**channel** is ready.

b) **Update**: Updating ~~involve~~**involves** setting up a new state then revoking the old one. ~~The party first commits~~**Both parties first commit** to a new state via the same mechanism as in the opening phase: sampling revocation keys (rk_A^n and rk_B^n respectively for the n^{th} state); exchanging their hashes; exchanging the signatures. Only then do they revoke the old state (here the $n - 1^{\text{th}}$ state) by disclosing to the other party their revocation keys (e.g., A sends rk_A^{n-1} to B).

If the steps were reversed, the party would have nothing to publish. The old state would be revoked, and the new one would not be signed. It could not close the channel if the counterparty stopped responding.

c) **Revocation**: A state is retired by handing its revocation key to the counterparty: Alice revokes state n by giving rk_A^n to Bob. The key is useless unless Alice later publishes that state, at which point it lets Bob take the whole channel balance. Revoking is therefore a proof of honesty, as publishing that old state could then result in punishment. ~~Thus, updating a state is done via committing to a new state by both parties, then revoking the previous state.~~

d) **Closing**: A channel can close in two ways. In the cooperative close, the parties sign one transaction together. This transaction sends the final balance directly to their wallets. Only one on-chain transaction is needed. There are no scripts and no delay. This is the cheapest outcome for both. In unilateral closing, either party can close. One party publishes its current commitment alone. The publisher's own output (out. [4]) is restricted by the CSV delay and by the revocation path. The party cannot spend its own output until the CSV delay ends. During that window, the counterparty can take it outright if the published state was revoked. The counterparty's (out. [3]) is spendable immediately. Therefore, unilateral ~~Closing~~**closing** disadvantages the publisher and favours the counterparty. This creates a punishment window, which is why an honest party closes unilaterally only when its counterparty has stopped responding.

Suggested last two sentences: That asymmetry is what makes punishment possible. It is also why an honest party closes unilaterally only when its counterparty has stopped responding — publishing alone is safe, but slow.

e) **Punishment**: If an old state is published on the blockchain by one party (i.e., a state whose revocation key is known to counterparty), the counterparty can defend themselves using the punishment paths shown in Fig. 2. Punishment works as an active defense: the defender must act (i.e., notice the fraud; send the ~~punishment~~**punishment** transaction; and wait for it to be accepted) within the window given by the CSV. Failure to react makes the protocol trivially insecure [21].

This is why our model must encode liveness: no agent can be ~~foreeibly~~**forcibly** put offline. We do so with restriction T3sec. IV-A and show in sec. V-B. This assumption is load-bearing. Removing it falsifies `Intermediary_Never_Loses_Under_Liveness`.

Suggested replacement for the last three sentences: Our model does not encode this liveness: the punishment rules are enabled as soon as a revoked state is published, so our punishment results assume the victim acts within its window (Sec. IV-A0a). The liveness we do encode concerns HTLC claiming (Restr. T3), and Sec. V-B shows it is load-bearing.

B. Multi-hop HTLC Routing

When the sender and recipients do not share a direct channel, the payment is routed through intermediaries along the path as in Fig. 3: it is a multi-hop payment. Because intermediaries are untrusted, each hop relay in an HTLC is conditioned on a single shared secret, the payment hash. As a result, the payment either completes along the entire path or fails. Furthermore, to provide incentive, the intermediaries are rewarded with a routing fee.

Example 1 (Path). We will use the running example where S sends money to R with the following path:

$$S \rightarrow F_1 \rightarrow \dots \rightarrow F_n \rightarrow R \quad (1)$$

a) **Locking**: For this, the receiver (R in Ex. 1) generates a preimage x and sends an invoice to the sender (S) containing the hash $y = H(x)$ and the requested amount v , keeping x strictly secret; the sender adds an HTLC to its channel with F_1 locking funds v_0 on that channel conditional on y before the deadline d_0 , and after d_0 the payer may reclaim it. The condition to unlock funds is to provide the preimage x' with $H(x') = y$ before the deadline (as in Fig. 2); This moves forward with each forwarder locking funds with the routing fee $v_{k-1} - v_k$ given to forwarder F_k . Each HTLC uses the same hash but with a decreasing deadline and funds.

$$v_0 > v_1 > \dots > v_n = v \quad (2)$$

$$d_0 > d_1 > \dots > d_n \quad (3)$$

At this ~~steps fund are~~**step, funds are** still locked by the payment hash; they have not moved yet. Each HTLC becomes an additional conditional output in the two parties' current commitment transaction, as explained in Sec. II-A. The intermediaries F_i can safely lock funds, since an incoming HTLC has a larger amount and ~~has~~ a later deadline **than its outgoing one**. If the outgoing HTLC is claimed, F_i learns x and still has time to unlock the incoming funds, keeping the difference as its fee.

b) **Release round**: In the redeem round, the receiver reveals x , the preimage, to the last forwarder to claim the final HTLC. Since every HTLC has the same preimage, each intermediary claims on its incoming channel using the same x , each learning x when the next hop claims from it. To claim v_{n-1} from F_{n-1} , F_n provides the secret x for its incoming HTLC, and it keeps the $v_{n-1} - v_n$ as a fee. The

Simon
meh explained

Sana
I divided it into two paras; it can be in one, idk

Claude
One paragraph reads better; keep the second as the REASON for the order. Invariant worth stating: at every instant each party holds an unrevoked, publishable state (BOLT 2: commitment signed before revoke_and_ack).

Claude
Duplicate of b) Update.

Claude
Logic: an honest party publishing its CURRENT state cannot be punished; what deters it is the CSV delay. The punishment window is what lets a CHEATER be punished.

Simon
Since we are more precise later, maybe we can move that in the "Liveness" section?

Claude
Missing space: T3 (Sec. IV-A). Contradiction: T3 encodes HTLC-claiming liveness, not punishment liveness; IV-A itself says punishment liveness is not encoded, and no restriction mentions Punish/Cheat/Publish/Revoked. Agree with Simon: move to Liveness.

Claude
Caption typo (Fig. 3): Payments → Payments.

Claude
Capital after semicolon.

Simon
todo better unify the two sections. This is repeating what is above; but I believe it might make more sense to have it here.

Claude
Agree: keep the operational description here (the running example lives here) and cut the duplicate in II-A.

preimage thus propagates backward hop by hop until the sender HTLC is settled. If any party in between fails to forward it and the timelock expires, the funds return to the party that locked them. If honest forwarders submit their claims on time, everyone gets paid; otherwise, no one does.

Remark 1 (Claiming and Liveness). *Claiming can happen by mutual negotiation or by unilateral closing. Notably, unilateral closing also publishes x as it is leaked by the closing transactions redeeming the HTLC.*

Remark 2 (Decreasing deadlines and Liveness). *Consider F_i holding an incoming HTLC at d_{i-1} and an outgoing HTLC at d_i . If F_{i+1} unilaterally claims the outgoing HTLC at d_i , F_i learns x as late as d_i and is left the window (d_i, d_{i-1}) to settle its incoming HTLC. The gap between d_{i-1} and d_i is the CLTV delta, which is precisely the time reserved for F_i to recover the funds.*

This means that, in case of unilateral close, forwarders need liveness in order to claim their incoming HTLC within their claim window [21].

[TODO] re-add the wormhole attack later. Likely when we talk about the model **[END TODO]**

Suggested two lines (the II opener already names the wormhole, so define it before first use): *The wormhole attack needs no party to go offline: two dishonest, non-adjacent parties relay x between themselves, skipping the honest intermediary in between, who does the work but earns no fee. We return to it, and to grieving, in Sec. V-C.*

III. SYMBOLIC MODEL AND NOTATION

We modeled our protocol using TAMARIN [16]. ~~It is~~ **It is a symbolic tool** for security-protocol analysis, which tracks a system represented as a multiset of rewriting facts—~~referred~~ **referred** to as state S . It evolves following user defined transition rules of the following shape:

$$[L] \text{ --- } [A] \longrightarrow [R] \quad (4)$$

with the premises (L), action facts (A), and conclusions (R). (4) can be triggered when L unifies with facts in S . It captures that L should be rewritten as R within S ~~while remembering the action facts~~ **while recording the action facts**. TAMARIN features linear and persistent facts; the former are consumed upon rule application, while the latter can be reused arbitrarily many times. The tool also provides built-in symbols $In()$, $Out()$ and $Fr()$ capturing respectively sending and ~~receiving~~ **receiving** over a public (i.e., attacker controlled) channel and generating fresh and unique value (e.g., for nonces). We further use the hashing, signing, and natural-numbers built-ins

Suggested fix: ...built-in facts $In()$ and $Out()$, capturing respectively receiving from and sending to a public (attacker-controlled) channel, and $Fr()$, generating fresh, unique values (e.g., nonces).

Suggested addition: *The first two supply the equational theories for $h(\cdot)$ and for $verify(sign(m, k), m, pk(k)) = true$,*

modelling the payment hash and the commitment signatures; the third supplies addition and comparison on naturals, used for amounts, fees and block heights. The N -hop theory replaces it with multiset (Sec. VI-B).

An execution produces a trace of action facts. Security goals are then first-order formulas over these trace, ~~enab~~ **namely** security queries—queries over all traces—, and reachability queries—ensuring a certain trace exists.

Missing definition (T1–T3 depend on it): TAMARIN also supports **restrictions**: trace formulas that discard every trace violating them. A restriction narrows the executions under consideration rather than stating a property to prove; T1–T3 (Sec. IV-A0a) are restrictions in this sense.

IV. FORMAL MODELING METHODOLOGY

A. Threat model and security goals

We assumed the standard Dolev-Yao adversary model [10] restricted to allow for liveness. It controls the whole network; it can read, modify, and inject messages of arbitrary complexity. It is however constrained to the defined term algebra, equational theory and added TAMARIN; in particular it cannot break cryptographic primitives, nor our liveness assumptions. We further modeled the compromised long-term key as a separate adversarial capability.

a) **Liveness:** As mentioned before, the security of the Lightning Network relies on the liveness of the agents, whether to ~~defer~~ **deter** cheating behaviour (Sec. II-A2e) or to claim HTLCs (Rmk. 1). A standard Dolev-Yao adversary has full control over the network (e.g., preventing it from noticing a fraud) or sending a message (e.g., sending the defending transaction), leading to trivial attacks on the Lightning Network [21].

To make the model relevant, we restrict the attacker's control over the network to allow for liveness. We also rule out pathological ~~honest~~ **honest** behaviour. Concretely, we impose the following restrictions, which constrain the routing layer. So, in our model, the punishment's liveness(sec. II-A2e) is not encoded but results when the counterparty acts within its window.

Restriction T1 (RedeemBeforeTimeout). *When a Redeems happens, it is before the timeout.*

Suggested: An HTLC can only be redeemed before it times out (clock well-formedness, not a liveness assumption).

Restriction T2a (ForwardTimeGap). *The outgoing HTLCs always timeouts after the incoming one.*

Suggested: If both HTLCs of a forwarding hop time out, the outgoing one times out first.

Restriction T2b (HonestPartiesActBeforeIncomingTimeout). *The outgoing HTLCs always timeouts after the incoming one.*

Suggested: If the incoming HTLC of a forwarding hop times out, the outgoing one has already timed out.

Restriction T3 (IntermediaryMustClaim). *An intermediary claims its incoming HTLC once the outgoing HTLC is redeemed.*

Sana
I commented it out...but I think if we refer it to that section in model two lines are ok...if you think it's ok add it

Claude
Answer to Sana: yes, two lines are enough; full treatment stays in V-C.

Simon
Not sure the "remembering the facts" is the right way to say it.

Claude
Suggest: 'while recording the action facts A in the trace'.

Claude
In()/Out() are reversed: In() is RECEIVING from the network, Out() is SENDING.

Simon
maybe point to

Claude
'and added Tamarin' seems garbled; 'and added Tamarin restrictions'?

Simon
this is already said before, so potentially we can remove.

Claude
Partly: II-A2e says punishment is active; it does not say the DY adversary can FORCE a party offline. Keep this sentence; it is what makes the restrictions necessary.

Simon
[for me] frame better

Claude
Good – this is correct and important. Suggest: '...is not encoded: our punishment results assume the counterparty acts within its window.'

Sana
I named it T1

Suggested: If a forwarder's outgoing HTLC is redeemed before its incoming one times out, the forwarder claims the incoming HTLC.

[NOTE] Just to make sure before I rewrite things:

- We encode things like the timeout choices with restrictions (T2*)
- We prevent Redeeming before a timeout via restriction (T1). Which sort of raises question marks to me look again at why we need it because you likely it to me at some point.
- We counter Rmk. 1 with Restr. T2b.
- We don't encode liveness of punishment (Sec. II-A2e) via restrictions.

Notably the last point is quite important because this is how this section is framed. I might have rewritten some stuff wrongly because I had the wrong mental model...

Also this is quite space inefficient. But I believe it is important enough to be worth the space. **[END NOTE]**

Answers to Simon's note (checked against multi-hop.spthy):

- T2* encode the timeout choices – yes.
- T1 is not a liveness assumption: $\text{Redeemed}(\text{ptr})@i \wedge \text{TimedOut}(\text{ptr})@j \Rightarrow i < j$ only forbids claiming an expired HTLC (clock well-formedness).
- Rmk. 1 is countered by T3, not T2b: T2b only orders the two timeouts (as Sec. V-B already argues).
- Correct: punishment liveness is not encoded; no restriction mentions Punish, Cheat or PublishRevoked.
- T2a is implied by T2b together with OneTimeoutPerPtr; drop it if the proofs do not slow down (to test).

[NOTE] V1 [END NOTE] Restr. T1 keeps the clock consistent, while Restr. T2a and T2b capture the CLTV-delta invariant. These three are structural. On the other hand, Restr. T3 is critically important for security, and Sec. V-B shows removing it falsifies `Intermediary_Never_Loses_Under_Liveness`. We proved the following properties: Payment Atomicity, Intermediary Safety, Routing Causality, and Value Conservation Sec. V-A, along with the timing results of Sec. V-B.

[NOTE] V2 [END NOTE] Restr. T1 to T2b are structural constraints of the CLTV clock and manage timing soundness. On the other hand, Restr. T3 is critically important for security and Sec. V-B shows removing it falsifies `Intermediary_Never_Loses_Under_Liveness`. We proved the following properties: Payment Atomicity, Intermediary Safety, Routing Causality, and Value Conservation Sec. V-A, along with the timing results of Sec. V-B.

B. Structural Limitations

The Lightning Network is a complex protocol. Verifying it as a single monolithic block quickly hits scalability problems with verification tools [8]; TAMARIN is no exception.

This is not an assumption but an observation. When different built-ins are combined in the TAMARIN, the search

space explodes. When the signature, equational theory, natural-number induction, and an unbounded clock are combined, the TAMARIN exhausts memory; any two of them remain traceable/tractable when combined. To overcome this problem, we split the model into multiple theories, each verified individually.

C. Soundness of the split

Splitting a model comes with limitations as well. We therefore acknowledge what our five theories do not establish. They are not a formal decomposition. In particular, there is no composition-theory/composition theorem to claim that the five theories together yield a property of one combined theory. Each theory is its own self-contained model. In table Table I, the part of the protocol each theory covers is stated. Each theory has its own lemmas, each proving a specific part of the protocol.

The theories share structure and it justifies reading them together.

[TODO] Explain why this is sound. This explanation likely needs to be made for each theory, so maybe it doesn't fit here.

[END TODO]

Suggested completion (answers the TODO: there are only two soundness questions, not one per theory): *The timing restrictions of the concrete theory are not stipulated in isolation. `Cltv.spthy` derives the CLTV-delta invariant over abstract block numbers and `Clock.spthy` checks that the channel lifecycle respects a block clock, so T1 and T2a follow from the clock model rather than being assumptions about behaviour. `timeout.spthy` exhibits the trace that T2b excludes, showing the restriction is not vacuous. T3 is different in kind: it is an obligation on an agent, and no ordering constraint can imply one. We keep it as an explicit assumption, and Sec. V-B shows it is load-bearing: removing it falsifies `Intermediary_Never_Loses_Under_Liveness`.*

The relation between the two routing theories is deliberately not a refinement: `Modif.spthy` is weaker than `multihop.spthy` in adversary — there is no Dolev-Yao network inside the routing abstraction — and stronger in path length; neither subsumes the other (Sec. VI).

Monotonicity of restriction. *Both linear-fact fixes of Sec. V-0a add a premise to a rule. Adding a premise can only shrink the set of substitutions under which a rule is enabled, so $\text{traces}(R') \subseteq \text{traces}(R)$: every all-traces lemma of R still holds of R', but exists-trace witnesses may be lost. Such a fix therefore cannot be checked locally, and every attack in this paper is stated as an exists-trace result: it must survive the fixes, not merely predate them.*

V. VERIFICATION OF THE CORE MODEL

[TODO] We may want to rename the files. `Modif.spthy` doesn't feel very professional ^^'. Also we are inconsistent in the file we name. **[END TODO]**

[NOTE] SJ: I feel like we need a paragraph to explain the table, but this one is rather weak. We likely want to explain in plainer english which file models and verifies what instead

Claude

Drop "the" before Tamarin (twice).

Claude

Do NOT write "each restriction the concrete theory assumes is discharged": false for T3 and contradicts V-B. Get Simon's OK on "no composition theorem".

Sana

I changed it a lil bit to avoid overdefining T1, but grouped 3 as structural. we have to keep one version either above or below.

Simon

see LaTeX

Claude

Keep V1: T1 is clock well-formedness, not a CLTV-delta constraint, so V2 over-groups. Fix 'security, and' and put the section fixes in parentheses.

Claude

Fix the inconsistency now (one file, three names: `multihop.spthy` / `multi-hop_modified.spthy` renaming `Modif.spthy` is optional (e.g. `routing3/routingN`).

of how many things it verifies. Potentially stating things like "multihop.spthy is our main file" [END NOTE]

Suggested table paragraph: *multihop.spthy is our main theory: it models the full channel lifecycle — opening, update, revocation, cooperative and unilateral closing, and punishment — composed with three-hop HTLC forwarding with per-hop fees, under the Dolev–Yao adversary. Everything we claim about atomicity, intermediary safety and fee conservation is proved there. The three small theories each isolate one thing multihop.spthy assumes: Cltv.spthy proves the CLTV-delta invariant over abstract block numbers, Clock.spthy checks the channel lifecycle against a block clock, and timeout.spthy exhibits the early-timeout race that T2b excludes. Modif.spthy asks whether the routing results depend on the path having length three; it replaces the network by an abstract routing fact and re-proves the path-independent properties for arbitrary N . In total the five theories discharge 87 lemma checks, 67 all-traces and 20 reachability.*

We verified 87 lemmas ~~across~~ across the ~~five~~ five theories shown in Table I. The concrete theory multihop_modified.spthy contributes 43 lemmas: 33 all-traces and 10 exists-trace witnesses (non-vacuity checks and the attack traces below); 5 of the all-traces lemmas are [reuse] helpers. The N -hop theory, Modif.spthy, re-verifies 26 of these multihop_modified.spthy lemmas over the abstract routing layer and adds 5 new to it, for 31 lemmas in total. The remaining lemmas come from the three small timing theories, Clock.spthy, Cltv.spthy, and timeout.spthy.

[TODO] Move to the right place. This is the old "fee" section from above. I believe it fits better here where we talk about the details of the mode. I am not sure where exactly yet though. [END TODO]

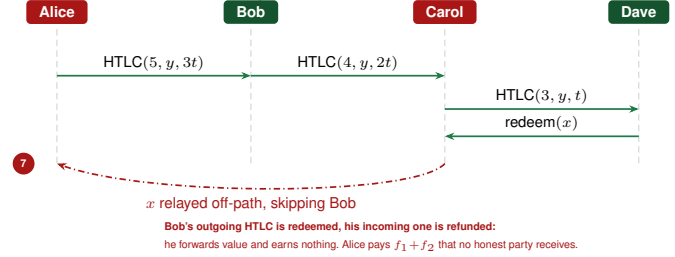
Our model includes the value of each HTLC and the fee at each hop. The reason is that the wormhole attack in sec. V-C steals routing fees. Amounts are natural-numbers terms, and each forwarding rule takes its outgoing amount and its fee from the network ($\text{In}(\% \text{fee}), \text{In}(\% \text{vOut})$), constrained only by the action fact $\text{Eq}(\% \text{vIn}, \% \text{vOut} \% + \% \text{fee})$. The fee is a per-hop quantity rather than Lightning's $\text{base_fee} + \text{fee_rate}$ formula. We can not prove anything about *fee policy*.

Suggested payoff sentence: *Since the adversary picks the fee at every hop, our results hold for every value-conserving fee schedule; what we cannot state is anything about fee policy.*

Fees are charged and earned by two separate events. FeeCharged is emitted when a forwarder locks its outgoing HTLC; FeeEarned is emitted only when it later claims its incoming one. The wormhole is exactly the gap between them, which is what Wormhole_Steals_Exactly_The_Fees measures.

We skip three things deliberately. Forward1_HTLC has no Wallet premise. So, a channel can carry any amount, so a forwarder never fails for lack of funds. The linear fact $\text{Free}(\text{ptr})$ is ~~spent~~ consumed on each added HTLC. There-

Alice and Carol are non-adjacent and collude; Bob is honest.



Alice learns x from Carol directly, so Bob never sees it.

Wormhole_Steals_Exactly_The_Fees shows the loss is exactly the path fees $f_1 + f_2$, not merely non-zero.

The two routing-layer failures, both reachable in multihop.spthy. Orange: benign (funds safe). Red: attack; red actors are dishonest.

Correction needed. The red annotation inside the figure — "Bob's outgoing HTLC is redeemed, his incoming one is refunded" — is exactly the situation Lemma 2 (Intermediary_Never_Loses_Under_Liveness) proves impossible without key compromise.

Wormhole_Fee_Theft_Reachable requires that Bob's outgoing HTLC is *never* redeemed: he is bypassed, not defrauded. Suggested annotation: *Bob's outgoing HTLC is never redeemed: he is bypassed, not defrauded. He locked capital and earns no fee, while the payment still settles end-to-end. Alice pays $f_1 + f_2$ that no honest party receives.* Also: "The two routing-layer failures" and "Orange: benign" are left over from when griefing was in this figure; it is now Fig. 3b.

Fig. 4. Wormhole

fore, a channel holds no more than one HTLC at a time. We also did not include onion routing. Each intermediary knows the full path explicitly (Sender, Forward1, Forward2). So, no claims about hiding who pays whom are made. The wormhole appears purely as fee theft; [6] also characterize it. Suggested: we do not capture the relationship-anonymity break that [6] also identify. StateA and StateB carry a state index and a revocation secret, not the output structure from Fig. 2. The two-stage HTLC claim becomes a single transition. The second stage adds a CSV delay and a revocation path. These are already enforced by Publish_Current and Claim_Funds (and Publish_Revoked for the revocation path).

```
1 rule Publish_Current_A:  
2 [ StateA($A, $B, ptr, n, sA, commitA, fA, fB),  
3 !ValidOnChain(ptr) ]  
4 -[ Publish($A, n, commitA),  
5 PublishState($A, n),  
6 OnChainDelayedProduce($A, n),  
7 OnChainInstantProduce($B, n) ] ->  
8 [ OnChainDelayed($A, n, commitA),  
9 OnChainInstant($B, n, commitA) ]
```

```
1 rule Claim_Funds_A:  
2 [ OnChainDelayed($A, n, txContent) ]  
3 -[ ClaimFunds($A, n), CSVPassed($A, n),  
4 SettledTx($A, txContent) ] ->  
5 [ FundsTo($A, n) ]
```

[NOTE] End orphaned pragraphs [END NOTE]

Fig. 4 shows the concrete 3-hop payment: lock, release, timeout refund, and wormhole. Each circled number matches a lemma group on the right that constrains that stage.

Stale: lock/release is now Fig. 3a, the refund Fig. 3b, the wormhole Fig. 4, and the badges index Table II (not a rail on the right). Suggested: Fig. 3a shows the successful three-hop payment, Fig. 3b the timeout refund and Fig. 4 the wormhole;

Simon
isn't this basically what is already written in the table?

Claude
Yes: with the paragraph above, the 87-lemmas paragraph is redundant; suggest deleting it.

Sana
Modified it; check it out. should we move it to Formal Modeling Methodology

Simon
Yes

Claude
Agree with Simon and Sana: move the fee/abstraction block to IV (it needs III's vocabulary; IV was too small).

Claude
two 'so'

Claude
these are \$Sender, \$Forward1, \$Forward2 in the rules

Claude
Malavolta et al. frame the wormhole as fee theft AND an anonymity break; we capture only the first.

Theory File	Aspect Modeled	Safety	Reachability	Builtins
multihop.spthy	Lifecycle, 3-hop routing, value/fees, wormhole	33	10	Hashing, S
Clock.spthy	Block-clock lifecycle & staggered-CLTV claim windows	6	3	Natural-nu
Cltv.spthy	Pure CLTV block arithmetic (delta positivity)	3	0	Natural-nu
timeout.spthy	Early-timeout race (T2b)	0	1	—
<i>Distinct properties</i>		42	14	
Modif.spthy	<i>Re-proof</i> of above over N -hop abstraction, fee/amount chaining	25	6	Hash, Sign
<i>Grand total (all 5 theories, 87 lemma checks)</i>		67	20	

The first four theories establish 56 distinct properties, and the N -hop theory adds a further 5, for 61 in total. Of the N -hop theory's 31 lemmas, 26 restate properties already proved in `multihop.spthy` over the abstract routing layer and 5 are new to it (generic forwarding causality and the unbounded-path value lemmas), giving 87 lemma checks in total.

TABLE I

THE FIVE-THEORY MODULAR ARCHITECTURE.

circled numbers index the lemma groups of Table II.

a) *The Linear Single-Spend Discipline:* We enforce the no-double-spend rule using linear facts. As mentioned before in Section III, a persistent fact $!F()$ is checked by a rule but not consumed, whereas a linear fact $F()$ is removed from the state when the rule fires.

In the Lightning network, the revocation secret is persistent knowledge ($!RevokedSecret$), and the revoked commitment output is a linear one-shot token ($RevokedCommitmentUnspent$), consumed when that state is published. Initially, the punishment rule only needed one thing to trigger: the persistent revocation secret alone:

```
1 rule Publish_Revoked_A: // WRONG: every premise persistent
2   [ !ChannelRecord(ptr,$A,$B), !ValidOnChain(ptr),
3     !RevokedSecret($A,n,s,c) ]
4   -[ PublishRevoked($A,n,c), Cheat($A,n) ] →
5     [ OnChainCheat($A,$B,n,c), RevealedSecret($A,n,s,c) ]
```

Here, punishment is gated only by persistent $!RevokedSecret$, the rule can fire again and again for the same revoked state. If a party cheats, it lets the honest party take the channel funds many times. Lemmas like `No_Punishment_Without_Cheating` still pass, because a cheat is still required. What they miss is that punishment should happen only once. To fix this issue, we split it into two separate rules: `Revoke_Old_Secret` still stores the secret as persistent $!RevokedSecret$. It also creates a linear token `RevokedCommitmentUnspent`.

```
1 rule Revoke_Old_Secret:
2   [ PendingA(...), PendingB(...), !Pk($A,pkA), !Pk($B,pkB)
3     ]
4   -[ StateUpdate($A,$B,ptr,n), Revoke($A,n_old,sAold) ] →
5     [ StateA(...), StateB(...),
6       !RevokedSecret($A,n_old,sAold,cA), // knowledge:
7       persistent
8       RevokedCommitmentUnspent(ptr,$A,$B,n_old,cA) ] //
9       output: linear
```

`Publish_Revoked` must consume that token, so the same revoked commitment can be punished only once:

```
1 rule Publish_Revoked_A: // FIXED: linear token gates the
2   spend
3   [ !ChannelRecord(ptr,$A,$B), !ValidOnChain(ptr),
4     !RevokedSecret($A,n,s,c),
5     RevokedCommitmentUnspent(ptr,$A,$B,n,c) ]
6   -[ PublishRevoked($A,n,c), Cheat($A,n) ] →
7     [ OnChainCheat($A,$B,n,c), RevealedSecret($A,n,s,c) ]
```

#	Group	Lemmas	Kind
1	Invoice & preimage	Invoice_Authenticates_Settlement, $\forall$ Preimage_Secret_Until_Released, Forged_Invoice_Requires_Key_Compromise, Invoice_Released_Once	$\forall$
2	Lock causality	Forward1_Requires_Offer, Forward2_Requires_Forward1, HTLC_On_Opened_Channel	$\forall$
3	Release causality	Fulfill_Requires_Forward2, Claim_Requires_Release, Settle_Requires_Receiver_Release	$\forall$
4	Atomicity	Payment_Atomicity_Under_Liveness, Settle_Excludes_Sender_Refund	$\forall$
5	non-vacuity	Multihop_Payment_Possible	$\exists$
5	Value & fee	EndToEnd_Value_Conservation, Fee_Conservation_Hop1/2, Fee_Strictly_Positive_Hop1/2, Receiver_Paid_Invoice_Amount	$\forall$
5	non-vacuity	Fees_Charged_On_Path_Possible	$\exists$
6	Refund / griefing	Refund_Requires_Timeout, Loss_Requires_Inaction	$\forall$
7	reachable	Honest_Intermediary_Refunds	$\exists$
7	Wormhole fee theft	Wormhole_Fee_Theft_Reachable, Wormhole_Steals_Exactly_The_Fees	$\exists$
8	Intermediary safety	Intermediary_Never_Loses_Under_Liveness, Timeout_Race_Blocked	$\forall$

$\forall$: all-traces safety; $\exists$: exists-trace witness.

TABLE II

LEMMA GROUPS OF `MULTIHOP.SPTHY`, KEYED TO THE CIRCLED NUMBERS IN FIG. 3A AND 4.

Persistent knowledge of the revocation secret is retained via $!RevokedSecret$. Only spending the funds is linear.

The same critical flaw we found in our initial model of the routing layer. With $!ChannelConnect$, one channel could add unboundedly many HTLCs on the same output. We fixed it by adding `Free(ptr)`. `Lock_Funds_And_Open` produced it once the channel is open and consumed when an HTLC is added on that channel.

```
1 rule Lock_Funds_And_Open:
2   [ ReadyToLock(...), Wallet($A,~fA), Wallet($B,~fB) ]
3   -[ ChannelOpen($A,$B,ptr), FundsLocked($A,$B,ptr) ] →
4     [ StateA(...), StateB(...), !ValidOnChain(ptr),
5       !ChannelConnect(ptr,$A,$B), !ChannelConnect(ptr,$B,$A)
6       , Free(ptr) ] // one-shot HTLC slot
```

Adding a linear premise can remove some exists-trace witnesses. So we re-ran the full concrete theory after the fix; all 43 lemmas still verify (the general argument is in Sec. IV-C).

Claude

This is the paper's central modelling lesson: consider a 'subsection' (it would also fix 'Section V-0a' in Fig. 1).

Claude

Table II: Row 1 marks Forged_Invoice_F as $\forall$, but it is exists-trace (your Appendix Remark 3): split it out as $\exists$. Caption: badge 6 is in Fig. 3b, so say 'Figs. 3 and 4'; group 8 has no badge.

Claude

Word order: 'We found the same critical flaw in our initial model of the routing layer.'

Claude

produces

A. Protocol Properties

We state the guarantees of the concrete theory as lemmas. Each is named after the TAMARIN property that establishes it. Adversarial capability is explicitly modeled as long-term key compromise (`Compromise_Ltk`). We apply rules to honest executions explicitly, not globally.

Lemma 1 (Payment Atomicity). *If both intermediaries forwarded an HTLC on hash y , the receiver's HTLC on y was redeemed, and no key was compromised, then the sender's HTLC on y cannot be refunded.*

```
1 lemma Payment_Atomicity_Under_Liveness:
2   "All F1 F2 ptrSF1 ptrF1F2 ptrF2R y #i #j #k.
3     ForwardHTLC(F1, ptrSF1, ptrF1F2, y) @ #i
4     & ForwardHTLC(F2, ptrF1F2, ptrF2R, y) @ #j
5     & Redeemed(ptrF2R, y) @ #k
6     & (not Ex P #l. LtkCompromised(P) @ #l)
7     ==> not (Ex #m. Refunded(ptrSF1, y) @ #m)"
```

Lemma 2 (Intermediary Safety). *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then that forwarder's long-term key was compromised.*

```
1 lemma Intermediary_Never_Loses_Under_Liveness:
2   "All P ptrIn ptrOut y #f #i #j.
3     ForwardHTLC(P, ptrIn, ptrOut, y) @ #f
4     & Redeemed(ptrOut, y) @ #i
5     & Refunded(ptrIn, y) @ #j
6     ==> Ex #r. LtkCompromised(P) @ #r"
```

Lemma 3 (Routing Causality). *Every forward is preceded by an upstream offer or by the predecessor's forward, carrying the same amount, unless the upstream party's key was compromised.*

```
1 lemma Forward1_Requires_Offer:
2   "All F1 F2 S ptrF1F2 y %vIn %vOut #i.
3     HTLCForwarded1(F1, F2, S, ptrF1F2, y, %vIn, %vOut)
4     @ #i
5     ==> (Ex ptrSF1 #j. HTLCOffered(S, F1, ptrSF1, y, %vIn)
6         @ #j & #j < #i)
7     | (Ex #k. LtkCompromised(S) @ #k & #k < #i)"
```

It is followed by `Forward2_Requires_Forward1` and `Fulfill_Requires_Forward2` in a 3-hop path. The same goes for `Forward1_Requires_Offer_Honest`, `Forward2_Requires_Forward1_Honest` and `Fulfill_Requires_Forward2_Honest`, with the additional restriction that the parties are honest (i.e., no key compromise). This is the "honest" version of the lemmas, which is what we need for the intermediary-safety proof.

Lemma 4 (Value Conservation). *At each hop, the incoming amount is the outgoing amount plus the fee charged there. End-to-end, the sender locks strictly more than the receiver's invoice amount.*

```
1 lemma Fee_Conservation_Hop1:
2   "All F1 F2 S p y %vIn %vOut %fee #i.
3     HTLCForwarded1(F1, F2, S, p, y, %vIn, %vOut) @ #i
4     & FeeCharged(F1, y, %fee) @ #i
5     ==> %vIn = %vOut + %fee"
```

Followed by `Fee_Conservation_Hop2`, `Fees_Charged_On_Path_Possible`, and lastly lemma `EndToEnd_Value_Conservation`. End-to-end conservation: over a full linked 3-hop payment, the sender

always locks the receiver's amount along with a strictly positive total fee. Value is neither created nor destroyed.

Suggested headline property (the paper's distinctive result; currently implicit): Fee integrity (informal). *On every settled payment, each intermediary that locked an HTLC on the path earns the fee it charged, and the sender pays exactly the invoice amount plus the fees earned. HTLC routing does not satisfy it: `Wormhole_Steals_Exactly_The_Fees` exhibits a settled payment in which `FeeCharged( $F_1, f_1$ )` and `FeeCharged( $F_2, f_2$ )` occur, the sender locks $v + f_1 + f_2$, and no `FeeEarned` occurs. We would state it as a named goal with a boxed formula, then show its violation. It is the value-carrying analogue of Smith et al.'s Path Integrity [22] — not a strengthening, since no containment argument is given.*

a) *Non-vacuity:* Safety lemmas are useless if the model cannot run at all, so we also checked exists-trace witnesses. `Multihop_Payment_Possible` shows that a full payment from the invoice to sender settlement is possible. `Distinct_Parties_Configuration` shows three channels among four different parties, with no key compromise and no dispute. `Fees_Charged_On_Path_Possible` shows two different intermediaries each taking a fee on the same payment, so the value lemmas are not only about a one-hop path.

B. Timing and T3

As discussed in Section IV-A, our model uses four timing restrictions (T1, T2a, T2b, and T3). `Cltv.spthy` derives the CLTV-delta invariant from concrete block numbers. `Clock.spthy` adds block-clock lifecycle safety. `timeout.spthy` shows the early-timeout race once T2b is removed. Together with Lemma: `Timeout_Race_Blocked` in the concrete theory `Multihop_modified`, this is a two-sided check: the race is reachable without T2b and blocked with it.

We also checked T3 behavior given T1/T2b. We removed T3 (`IntermediaryMustClaim`) and re-ran. `Intermediary_Never_Loses_Under_Liveness` failed, and Tamarin found a counterexample. T2b only orders the two timeouts; it does not force the intermediary to claim its incoming HTLC. That claim is exactly T3, and no ordering rule can force an action. So T3 is load-bearing for intermediary safety. We did not make claims for T1, T2a, or T2b as for T3. We therefore only checked T3 because it is the only one of the four that is an obligation, not an ordering.

Suggested figure: the T3 counterexample is the load-bearing claim of this subsection but is only asserted. A hand-drawn 4-arrow sequence (forwarder's outgoing HTLC redeemed; forwarder never claims; incoming times out and is refunded; forwarder loses v_{i-1}) read off the Tamarin trace would make it checkable.

C. Griefing and Wormhole

`Honest_Intermediary_Refunds` shows an honest intermediary locking capital and then refunding on timeout, with no settlement and no key compromise. The funds are still refundable, so safety still holds. We show this trace as a reachable griefing DoS, not as a new attack. The wormhole

Claude

Precision: End-ToEnd_Value_Co only proves Ex %d. %vS = %v %d (sender locks strictly more). That the surplus follows from Fee_Conservation Also Fees_Charged_On is exists-trace, not a conservation lemma.

Claude

Name it and it becomes the paper's punchline; LeanDY explicitly excludes fees.

Claude

Distinct_Parties is not in Table II; add it or drop the mention.

Claude

T2a is implied by T2b + One TimeoutPerPtr: say so, or drop T2a after testing.

Claude

'Lemma: stray; 'theoryMulti-hop_modified' needs a space and should read multihop.spthy (the file name used everywhere else).

Claude

Two "still" in one sentence.

attack [6] is an instance of the agent-skipping attack formalized symbolically by [22]. It is a known attack, so we do not claim it as new. **We include it for two reasons.** First, after the linear single-spend fix, it is a soundness check: the model must still admit this behavior, or the fix would be too strong. Second, our value layer lets us measure the loss. `Wormhole_Fee_Theft_Reachable` shows a payment that settles while the middle hop is never redeemed and earns no fee. `Wormhole_Steals_Exactly_The_Fees` pins that loss to exactly the path fees. Both appear as (6) and (7) in Figure 4.

Suggested modelling note (a reviewer will ask how the attack arises with no corrupted party): *In `multihop.spthy`, `Sender_Settle` consumes an unauthenticated `In(<'fulfill', ptr, x>)`. Once the receiver's fulfilment puts `x` on the network, the Dolev-Yao adversary can inject it on the sender's channel, so the attack needs no key compromise. The network adversary thus stands in for out-of-band collusion between two dishonest, non-adjacent intermediaries — the real-world wormhole — which authenticated channels do not prevent.*

Positioning vs [22]: our fee result is the value-carrying analogue of Path Integrity; in their Table 3 the Lightning unlock phase already fails Path Integrity, so the novelty is measuring the loss, not detecting the skip.

VI. SCALING TO N HOPS

The concrete theory fixes the path at three hops. The obvious next question is whether these safety rules work for routes of any length. The answer was not simple by simply adding rules. Also, more steps mean more options for attackers, and TAMARIN gets stuck in a loop.

A. Abstracting the Routing Layer

The first problem was the signed HTLC offer. When tracing a forwarded message, Tamarin treats every party as a potential sender. With an N -hop, the analysis loops infinitely. To solve this, we swapped the public message for a private, single-use fact passed to the next intermediary.

```
1 // GENERIC FORWARD i.e one rule, any number of hops.
2 rule Forward_HTLC:
3   [ Route($Prev, $Me, ptrIn, y, amtOut + fee),
4     !ChannelConnect(ptrOut, $Me, $Next),
5     Free(ptrOut) ]
6   -[ HTLCForwarded($Me, $Prev, $Next, ptrIn, ptrOut, y,
7     amtOut + fee, amtOut),
8     HTLCAdded(ptrOut, y),
9     ForwardHTLC($Me, ptrIn, ptrOut, y),
10    FeeExpected($Me, ptrIn, ptrOut, y, fee),
11    Neq(ptrIn, ptrOut), Neq($Prev, $Me), Neq($Me, $Next)
12  ] →
13  [ IntermediaryPending($Me, $Prev, $Next, ptrIn, ptrOut,
14    y,
15    amtOut + fee, amtOut, fee),
16    Route($Me, $Next, ptrOut, y, amtOut) ]
```

This generic rule applies to each hop regardless of the path length. Because the HTLC is no longer a recursive signed message, TAMARIN's origin analysis no longer loops.

B. Fee Chaining Across Hops

In VI-A, `Route` carries the outgoing amount into the next hop. Each forward therefore enforces

$$v_{\text{in}} = v_{\text{out}} + \text{fee}.$$

The receiver is paid the invoice amount; everything above that is path fees. So end-to-end value conservation is a property of the unbounded model. So the value lemmas thus hold for arbitrary N , not only for the fixed 3-hop theory.

a) **Honest Scope of the Abstraction:** The Dolev-Yao network adversary is no longer in the N -hop model because the path is inside the `Route` fact. So it can no longer inject, replay, or forge messages. Therefore, N -hop theory does not claim security against network attacks. It only shows that the internal N -hop state machine stays sound for arbitrary N . This includes timeouts, preimage propagation, and local transitions.

b) **Why the wormhole disappears in N -hop:** The wormhole is not reachable in the abstract model, and this is structural rather than accidental. Skipping a hop means sending (x) around the honest intermediary. In the concrete model, two dishonest non-adjacent parties get that path from the Dolev-Yao network adversary. In `Route`, there is no such path. As a result, the attack cannot occur.

[NOTE] These last two points are a bit annoying. I'll see how I can spin them. **[END NOTE]**

Suggested spin (Simon's NOTE): make the disappearance a controlled experiment. *Why the wormhole disappears is also testable. We add to `Modif.spthy` a single labelled rule that re-introduces exactly the capability the abstraction removes — relaying a fulfilment between two channels that are not adjacent on the path:*

```
rule AttackerSkip:
  [ Fulfill(ptrOut, x), !ChannelConnect(ptrIn, P1, P2) ]
  -[ Skipped(ptrIn, ptrOut), Neq(ptrIn, ptrOut) ] →
  [ Fulfill(ptrIn, x) ]

restriction SkipOnce:
  "All a1 b1 a2 b2 #i #j.
   Skipped(a1,b1) @ #i & Skipped(a2,b2) @ #j ==> #i = #j"
```

With the rule, the wormhole lemma — now requiring a `Skipped` event — becomes reachable; without it, it is not, while `Multihop_Payment_With_Fees_Possible` verifies in both runs (to confirm: quote both —prove summaries). The immunity of the abstraction is therefore attributable to exactly one missing capability rather than to an artefact of the encoding. We do not read the reachable trace as a new attack: the relay stands in for out-of-band collusion between two dishonest, non-adjacent intermediaries, which authenticated channels do not prevent; it is not a man-in-the-middle, which BOLT 8's authenticated transport would prevent.

VII. RELATED WORK

Formal analyses of payment channels. Grundmann and Hartenstein [8] model-check a stepwise refinement of Lightning in TLA+, which bounds the explored state space. Stefanski and Litos [9] present a formally verified Lightning Network (to check scope). LeanDY [23], from our group, mechanises

Claude
Precisely: the middle hop's OUTGOING HTLC is never redeemed (that is the lemma's condition).

Claude
(6) is in Fig. 3b, (7) in Fig. 4.

Claude
"The answer was not simple by simply adding rules" — rephrase: "Simply adding rules does not work."

Claude
"With an N -hop" → "With an N -hop path".

Claude
"So ... So ... thus" is repetitive: e.g. "End-to-end value conservation is therefore a property of the unbounded model, and the value lemmas hold for arbitrary N ."

Claude
Needs SkipOnce: without it the rule has Fulfill on both sides and the backward search regresses forever. Also check that the induction lemmas still terminate.

Claude
Exists-trace caveat: over-approximating the adversary weakens a friend attack

a Dolev–Yao analysis in Lean; its Lightning study explicitly excludes fees and states that liveness is out of reach of standard Dolev–Yao models, which are precisely the two aspects treated here. Aumayr et al. [7] generalise payment channels.

Attacks and liveness. Tikhomirov et al. [5] give a quantitative analysis of Lightning security and anonymity. Malavolta et al. [6] introduce the wormhole attack and anonymous multi-hop locks; their attack both steals fees and breaks relationship anonymity, and our model, which has no onion routing, captures only the fee part. McCorry et al. [21] (Pisa) show that watchtowers can outsource the active defence that punishment requires.

Agent skipping. Smith et al. [22] define Path Integrity; in their analysis Lightning’s unlock phase does not satisfy it. Our fee result is a value-carrying analogue of Path Integrity, not a strengthening, since no containment argument has been established.

Tools. Symbolic tools (ProVerif [15], Tamarin [16], used e.g. for TLS 1.3 [18]) contrast with computational ones [11], [12], [13], [14]. We chose Tamarin for unbounded sessions, for multiset rewriting, which represents on-chain state naturally as linear and persistent facts, and for its built-in natural numbers.

VIII. DISCUSSION AND LIMITATIONS

Liveness. Intermediary safety rests on one assumption, Restr. T3: a forwarder whose outgoing HTLC is redeemed claims its incoming one in time. It is load-bearing: without it, `Intermediary_Never_Loses_Under_Liveness` fails with a counterexample. Punishment liveness is not encoded; no restriction mentions `Punish`, and `Punish_A/Punish_B` are merely enabled once an on-chain cheat exists.

Soundness of the split. There is no composition theorem between the five theories. `Cltv.spthy` and `Clock.spthy` justify Restr. T1 and T2a from a block clock, `time-out.spthy` shows Restr. T2b is non-vacuous, and Restr. T3 remains an explicit assumption. The N-hop theory is not a refinement of the 3-hop one: it has a weaker adversary (no Dolev–Yao network inside the route) but arbitrary path length.

Abstractions. We abstract the fee formula, on-chain fees, channel capacity, onion routing and the two-stage HTLC claim, and allow one HTLC per channel (via `Free(ptr)`). As noted in Sec. VI, `Settle_Requires_Receiver_Release` does not terminate in the N-hop variant.

Modelling lesson. Encoding an on-chain output as a persistent instead of a linear fact can leave all safety lemmas green; only exists-trace witnesses expose the error (rule concerned: to confirm). Adding a premise shrinks the trace set, so all-traces lemmas survive while witnesses may vanish; each fix therefore requires a full re-run.

IX. CONCLUSION

We presented a Tamarin analysis of Lightning payment channels and multi-hop HTLC routing, split into five theories totalling 87 lemma checks. It establishes atomicity, value conservation (the sender locks strictly more than the invoice)

and intermediary safety under an explicit, load-bearing liveness assumption, and exhibits the wormhole fee theft as a reachable trace. Future work: encoding punishment liveness, multiple HTLCs per channel, onion routing and anonymity, a composition argument for the split, and BOLT-level transaction structure.

REFERENCES

- [1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” p. 21260, 2008.
- [2] J. Poon and T. Dryja, “The bitcoin lightning network: Scalable off-chain instant payments,” 2016.
- [3] (2026) Lightning Network Statistics | IML - Lightning Network Search and Analysis Engine - Bitcoin mainnet. [Online]. Available: <https://web.archive.org/web/20260810134919/https://1ml.com/statistics>
- [4] “Lightning/bolts,” Lightning Network, 2026. [Online]. Available: <https://github.com/lightning/bolts>
- [5] S. Tikhomirov, P. Moreno-Sanchez, and M. Maffei, “A Quantitative Analysis of Security, Anonymity and Scalability for the Lightning Network,” in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2020, pp. 387–396.
- [6] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, “Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability,” in *Proceedings 2019 Network and Distributed System Security Symposium*. Internet Society, 2019.
- [7] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, “Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures,” in *Advances in Cryptology – ASIACRYPT 2021*, M. Tibouchi and H. Wang, Eds. Springer International Publishing, 2021, vol. 13091, pp. 635–664.
- [8] M. Grundmann and H. Hartenstein, “Security of the lightning network: Model checking a stepwise refinement with TLA+,” in *Integrated Formal Methods*, F. Damiani and M. Farrell, Eds. Springer Nature Switzerland, 2026, pp. 313–335.
- [9] G. , R. Stefański, and O. S. T. Litos, “A Formally Verified Lightning Network,” in *Financial Cryptography and Data Security*, C. Garman and P. Moreno-Sanchez, Eds. Springer Nature Switzerland, 2026, pp. 3–20.
- [10] D. Dolev and A. Yao, “On the security of public key protocols,” vol. 29, no. 2, pp. 198–208, 1983.
- [11] B. Blanchet, “A computationally sound mechanized prover for security protocols,” in *2006 IEEE Symposium on Security and Privacy (S&P’06)*, 2006, pp. 15 pp.–154.
- [12] G. Barthe, B. Grégoire, S. Heraud, and S. Z. Béguelin, “Computer-Aided Security Proofs for the Working Cryptographer,” in *Advances in Cryptology – CRYPTO 2011*, ser. Lecture Notes in Computer Science, P. Rogaway, Ed. Springer, 2011, pp. 71–90.
- [13] D. Baelde, S. Delaune, C. Jacomme, A. Koutsos, and S. Moreau, “An Interactive Prover for Protocol Verification in the Computational Model,” in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 537–554.
- [14] S. Jeanteur, L. Kovács, M. Maffei, and M. Rawson, “CryptoVampire: Automated Reasoning for the Complete Symbolic Attacker Cryptographic Model.” IEEE Computer Society, 2024, pp. 3165–3183.
- [15] B. Blanchet, “An Efficient Cryptographic Protocol Verifier Based on Prolog Rules,” in *14th IEEE Computer Security Foundations Workshop (CSFW-14)*. IEEE Computer Society, 2001, pp. 82–96.
- [16] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, N. Sharygina and H. Veith, Eds. Springer, 2013, pp. 696–701.
- [17] B. Lipp, B. Blanchet, and K. Bhargavan, “A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol,” in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019, pp. 231–246.
- [18] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, “A Comprehensive Symbolic Analysis of TLS 1.3,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’17. Association for Computing Machinery, 2017, pp. 1773–1788.

Claude

'p. 21260'
looks wrong
for the Bitcoin
whitepaper.

Claude

Author list
broken: first
author missing
(‘G. , R.’). Fix
the .bib author
field.

- [19] G. Andresen, “BIP 11: M-of-N Standard Transactions,” 2011-10, 2011. [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0011.mediawiki>
- [20] E. Lombrozo, J. Lau, and P. Wuille, “BIP 141: Segregated witness (consensus layer),” 2015-12, 2015. [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>
- [21] P. McCorry, S. Bakshi, I. Bentov, S. Meiklejohn, and A. Miller, “Pisa: Arbitration Outsourcing for State Channels,” in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, ser. AFT ’19. Association for Computing Machinery, 2019, pp. 16–30.
- [22] Z. Smith, H. Jonker, S. Mauw, and H. Lee. (2022) Modelling Agent-Skipping Attacks in Message Forwarding Protocols.
- [23] [Authors to fill in], “LeanDY ...,” arXiv:2607.03406, 2026. (suggested: not yet cited)
- [24] J. A. Donenfeld and K. Milner, “Formal Verification of the WireGuard Protocol,” technical report, 2017. (suggested: the Tamarin model of WireGuard)

APPENDIX A LEMMA CATALOGUE

[TODO] SJ: This was LLM generated. We need to unloppify it or remove it all together by submission date. I added it mostly for my own referene. **[END TODO]**

This appendix renders every lemma and query of Table II in natural language and gives its exact Tamarin statement, so that the table can be checked against the theories.

Unless stated otherwise, every lemma below is verified in `multiHop_modified.spthy` (the concrete three-hop theory of Sec. V-A). The table’s caption says “`multiHop.spthy`” because that is the ancestral name of the file; the concrete theory currently contributing these lemmas is `multiHop_modified.spthy`, and the two variants of the atomicity/intermediary lemmas named with the suffix `_Liveness` (rows 4 and 8) exist under exactly those names only there. We give line numbers in that file. We use the same naming convention as the table: A = all-traces safety property, E = exists-trace witness (reachability / non-vacuity).

Throughout, action facts carry their intended meaning from Sec. III: `InvoiceCreated` — the receiver issued an invoice with a given payment hash and amount; `HTLCOffered` / `HTLCForwarded1` / `HTLCForwarded2` — the corresponding party locked an HTLC (offered or forwarded) with an incoming and an outgoing amount; `HTLCFulfilled` — the receiver redeemed its final HTLC; `SenderSettled` — the sender settled its HTLC with a preimage; `IntermediaryClaimed` — an intermediary claimed its incoming HTLC; `Redeemed` / `Refunded` — the HTLC on a given channel was settled (payment) or timed out (refund); `TimedOut` — a channel timed out; `ForwardHTLC` — an intermediary forwarded an HTLC from an incoming to an outgoing channel on the same hash; `PreimageReleased` / `SecretPreimage` — the receiver released / privately holds a preimage; `FeeCharged` / `FeeEarned` — an intermediary charged / earned a fee on a hash; `ReceiverPaid` — the receiver was credited an amount; `LtkCompromised` — a party’s long-term key was compromised; `K(x)` — the adversary knows the term `x`.

A. Invoice and Preimage

Lemma 5 (`Invoice_Authenticates_Settlement` ($\forall$)). *The sender can settle an HTLC only if the payment*

hash corresponds to an invoice the receiver actually issued, at a strictly earlier time; the only way around this is for the receiver’s long-term key to have been compromised (strictly earlier as well). In short, a settlement is legitimate only when a genuine invoice backs it; a settlement without such an invoice is attributable to a compromised receiver.

```
691 lemma Invoice_Authenticates_Settlement:
692   "All S Fl R ptr x #i.
693   SenderSettled(S, Fl, R, ptr, x) @ #i
694   ==> (Ex %v #j. InvoiceCreated(R, S, h(x), %v) @ #j & #
        j < #i)
695   | (Ex #r. LtkCompromised(R) @ #r & #r < #i) "
```

Lemma 6 (`Preimage_Secret_Until_Released` ($\forall$)). *As long as the receiver holds a secret preimage and has not released it, the adversary does not know it: the first moment the adversary can learn the preimage is strictly after the receiver has released it. Formally, if the receiver generated a secret preimage and the adversary later learns it, then the receiver must have released that preimage at a strictly earlier point in the trace.*

```
612 lemma Preimage_Secret_Until_Released:
613   "All R x #i #j.
614   SecretPreimage(R, x) @ #i
615   & K(x) @ #j
616   & #i < #j
617   ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #j "
```

Lemma 7 (`Forged_Invoice_Requires_Key_Compromise` ($\exists$)). *The sender can settle an HTLC even though the receiver never created the corresponding invoice — but only in traces in which the receiver’s long-term key was compromised. This is the attack trace showing that a forged invoice cannot be cashed in without breaking the receiver.*

```
698 lemma Forged_Invoice_Requires_Key_Compromise:
699   exists-trace
700   "Ex S Fl R ptr x #i #r.
701   SenderSettled(S, Fl, R, ptr, x) @ #i
702   & LtkCompromised(R) @ #r
703   & not (Ex %v #j. InvoiceCreated(R, S, h(x), %v) @ #j) "
```

Remark 3. *In the theory this lemma is declared exists-trace (kind E), whereas Table II groups it with all-traces lemmas (kind A). The intended reading is the reachability one above; if the table is meant to report an all-traces guarantee, that guarantee is in fact already the contrapositive of `Invoice_Authenticates_Settlement`. The classification in the table should be corrected to E.*

Lemma 8 (`Invoice_Released_Once` ($\forall$)). *The receiver releases a given preimage at most once: if the same receiver releases the same preimage at two times in a trace, those two times coincide, i.e. there is a single release event. This holds because the preimage is fresh per invoice, and even if the adversary replays a public fulfill message it cannot make the receiver release the preimage a second time.*

```
568 lemma Invoice_Released_Once [reuse]:
569   "All R x #i #j.
570   PreimageReleased(R, x) @ #i
571   & PreimageReleased(R, x) @ #j
572   ==> #i = #j "
```

Claude
No venue: add arXiv:2201.08686 (or the published version).

Claude
Suggest: keep a trimmed version in the artifact README; in the paper keep only what Table II needs. (Typo in TODO: "reference".)

Claude
Fix the file name instead of documenting the inconsistency: a reviewer sees two names for one file.

Claude
Good catch — now fix Table II row 1.

B. Lock Causality

Lemma 9 (Forward1_Requires_Offer ($\forall$)). *The first intermediary can lock (forward) an HTLC on its outgoing channel only if the sender previously offered it an HTLC with the same payment hash and the same incoming amount on its incoming channel, at a strictly earlier time — unless the sender's key was compromised. In other words, forwarding is causally preceded by an upstream offer.*

```
635 lemma Forward1_Requires_Offer:
636   "All F1 F2 S ptrF1F2 y %vIn %vOut #i.
637     HTLCForwarded1(F1, F2, S, ptrF1F2, y, %vIn, %vOut) @ #i
638   ==> (Ex ptrSF1 #j. HTLCOffered(S, F1, ptrSF1, y, %vIn)
639     @ #j & #j < #i)
640   | (Ex #k. LtkCompromised(S) @ #k & #k < #i)"
```

Lemma 10 (Forward2_Requires_Forward1 ($\forall$)). *The second intermediary can forward an HTLC to the receiver only if the first intermediary previously forwarded an HTLC with the same hash and with a matching amount, at a strictly earlier time — unless the first intermediary's key was compromised.*

```
641 lemma Forward2_Requires_Forward1:
642   "All F2 R F1 ptrF2R y %vIn %vOut #i.
643     HTLCForwarded2(F2, R, F1, ptrF2R, y, %vIn, %vOut) @ #i
644   ==> (Ex S ptrF1F2 %vPrev #j.
645     HTLCForwarded1(F1, F2, S, ptrF1F2, y, %vPrev, %vIn)
646     @ #j & #j < #i)
647   | (Ex #k. LtkCompromised(F1) @ #k & #k < #i)"
```

Lemma 11 (HTLC_On_Opened_Channel ($\forall$)). *An HTLC is only ever added on a channel that has been opened beforehand: whenever an HTLC is added on a channel, two parties opened that channel at a strictly earlier time.*

```
624 lemma HTLC_On_Opened_Channel [reuse, use_induction]:
625   "All ptr y #i.
626     HTLCAdded(ptr, y) @ #i
627   ==> Ex X Y #j. ChannelOpen(X, Y, ptr) @ #j & #j < #i"
```

C. Release Causality

Lemma 12 (Fulfill_Requires_Forward2 ($\forall$)). *The receiver can fulfill (redeem) an HTLC only if the second intermediary previously forwarded it an HTLC with the same hash and the same amount on the same channel, at a strictly earlier time — unless that intermediary's key was compromised.*

```
648 lemma Fulfill_Requires_Forward2:
649   "All R F2 ptrF2R y %v #i.
650     HTLCFulfilled(R, F2, ptrF2R, y, %v) @ #i
651   ==> (Ex F1 %vIn #j. HTLCForwarded2(F2, R, F1, ptrF2R,
652     y, %vIn, %v) @ #j & #j < #i)
653   | (Ex #r. LtkCompromised(F2) @ #r & #r < #i)"
```

Lemma 13 (Claim_Requires_Release ($\forall$)). *An intermediary can claim its incoming HTLC with a preimage only after (strictly later than) the receiver has released that preimage: the release of the preimage is a prerequisite for any intermediary settlement.*

```
678 lemma Claim_Requires_Release:
679   "All MidP InP ptrIn R x #i #j.
680     IntermediaryClaimed(MidP, InP, ptrIn, x) @ #i
681     & SecretPreimage(R, x) @ #j
682   ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```

Lemma 14 (Settle_Requires_Receiver_Release ($\forall$)). *When the sender settles an HTLC on a genuine invoice,*

the receiver must have released the corresponding preimage strictly earlier: a settlement never precedes the receiver's release of the preimage.

```
629 lemma Settle_Requires_Receiver_Release:
630   "All S F1 R ptr x %v #i #j.
631     SenderSettled(S, F1, R, ptr, x) @ #i
632     & InvoiceCreated(R, S, h(x), %v) @ #j
633   ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```

D. Atomicity

Lemma 15 (Payment_Atomicity_Under_Liveness ($\forall$)). *If both intermediaries have forwarded the HTLCs that chain a three-hop payment (locking it end-to-end), the receiver's final HTLC has been redeemed, and no party's long-term key is compromised, then the sender's HTLC can never be refunded: once the payment is locked all the way through and released at the receiver, the sender's money is spent for good. This is the atomicity of the payment under the liveness restrictions.*

```
730 lemma Payment_Atomicity_Under_Liveness:
731   "All F1 F2 ptrSF1 ptrF1F2 ptrF2R y #i #j #k.
732     ForwardHTLC(F1, ptrSF1, ptrF1F2, y) @ #i
733     & ForwardHTLC(F2, ptrF1F2, ptrF2R, y) @ #j
734     & Redeemed(ptrF2R, y) @ #k
735     & (not Ex P #l. LtkCompromised(P) @ #l)
736   ==> not (Ex #m. Refunded(ptrSF1, y) @ #m)"
```

Lemma 16 (Settle_Excludes_Sender_Refund ($\forall$)). *If the sender has settled an HTLC with a preimage, then that same HTLC can never be refunded later: settlement and refund of the sender's HTLC are mutually exclusive.*

```
685 lemma Settle_Excludes_Sender_Refund:
686   "All S F1 R ptr x #i.
687     SenderSettled(S, F1, R, ptr, x) @ #i
688   ==> not (Ex #j. HTLCRefunded(S, ptr, h(x)) @ #j)"
```

Lemma 17 (Multihop_Payment_Possible ($\exists$)). *Non-vacuity witness. There exists a complete trace realizing a successful three-hop payment, in order: invoice creation, the sender's offer, both forwards, the receiver fulfilling its HTLC, both intermediaries claiming their incoming HTLCs, and the sender's final settlement. This shows the safety lemmas speak about a protocol that can actually run.*

```
574 lemma Multihop_Payment_Possible:
575   exists-trace
576   "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R x %v %vS %vMid
577     #t1 #t2 #t3 #t4 #t5 #t6 #t7 #t8.
578     InvoiceCreated(R, S, h(x), %v) @ #t1
579     & HTLCOffered(S, F1, ptrSF1, h(x), %vS) @ #t2
580     & HTLCForwarded1(F1, F2, S, ptrF1F2, h(x), %vS, %vMid) @ #t3
581     & HTLCForwarded2(F2, R, F1, ptrF2R, h(x), %vMid, %v) @ #t4
582     & HTLCFulfilled(R, F2, ptrF2R, h(x), %v) @ #t5
583     & IntermediaryClaimed(F2, F1, ptrF1F2, x) @ #t6
584     & IntermediaryClaimed(F1, S, ptrSF1, x) @ #t7
585     & SenderSettled(S, F1, R, ptrSF1, x) @ #t8
586     & #t1 < #t2 & #t2 < #t3 & #t3 < #t4 & #t4 < #t5
587     & #t5 < #t6 & #t6 < #t7 & #t7 < #t8"
```

E. Value and Fee

Lemma 18 (Fee_Conservation_Hop1 ($\forall$)). *At the first forwarding step, the amount the first intermediary receives on its incoming HTLC equals the amount it locks on its outgoing*

HTLC plus the fee it charges at that hop. Value is conserved per hop.

```
752 lemma Fee_Conservation_Hop1:
753   "All F1 F2 S p y %vIn %vOut %fee #i.
754   HTLCForwarded1(F1, F2, S, p, y, %vIn, %vOut) @ #i
755   & FeeCharged(F1, y, %fee) @ #i
756   ==> %vIn = %vOut %+ %fee"
```

Lemma 19 (Fee_Conservation_Hop2 (V)). *Same as Fee_Conservation_Hop1, for the second forwarding step: the second intermediary's incoming amount equals its outgoing amount plus the fee it charges there.*

```
758 lemma Fee_Conservation_Hop2:
759   "All F2 R F1 p y %vIn %vOut %fee #i.
760   HTLCForwarded2(F2, R, F1, p, y, %vIn, %vOut) @ #i
761   & FeeCharged(F2, y, %fee) @ #i
762   ==> %vIn = %vOut %+ %fee"
```

Lemma 20 (Fee_Strictly_Positive_Hop1 (V)). *The routed amount strictly decreases at the first forward: the incoming amount is the outgoing amount plus a strictly positive delta, i.e. the intermediary never forwards more value than it receives and always takes a strictly positive fee.*

```
821 lemma Fee_Strictly_Positive_Hop1:
822   "All F1 F2 S p y %vIn %vOut #i.
823   HTLCForwarded1(F1, F2, S, p, y, %vIn, %vOut) @ #i
824   ==> Ex %d. %vIn = %vOut %+ %d"
```

Lemma 21 (Fee_Strictly_Positive_Hop2 (V)). *Same as Fee_Strictly_Positive_Hop1, for the second forward: the routed amount strictly decreases by a strictly positive delta.*

```
826 lemma Fee_Strictly_Positive_Hop2:
827   "All F2 R F1 p y %vIn %vOut #i.
828   HTLCForwarded2(F2, R, F1, p, y, %vIn, %vOut) @ #i
829   ==> Ex %d. %vIn = %vOut %+ %d"
```

Suggested addition (Lemmas 20–21): Tamarin's natural-numbers builtin has no zero (its smallest term is 1), so `Ex %d. %vIn = %vOut %+ %d` already forces $vIn > vOut$.

Lemma 22 (EndToEnd_Value_Conservation (V)). *Across a full linked three-hop payment, the sender locks exactly the receiver's amount plus a (strictly positive) total fee: value is neither created nor destroyed along the whole path.*

```
834 lemma EndToEnd_Value_Conservation:
835   "All S F1 F2 R pS pF pR y %vS %vM %v #o #f1 #f2.
836   HTLCOffered(S, F1, pS, y, %vS) @ #o
837   & HTLCForwarded1(F1, F2, S, pF, y, %vS, %vM) @ #f1
838   & HTLCForwarded2(F2, R, F1, pR, y, %vM, %v) @ #f2
839   ==> Ex %d. %vS = %v %+ %d"
```

Lemma 23 (Receiver_Paid_Invoice_Amount (V)). *The receiver is paid exactly the amount it invoiced: whenever the receiver is credited for a payment hash, that hash corresponds to an invoice it created, with precisely this amount, at an earlier time.*

```
764 lemma Receiver_Paid_Invoice_Amount:
765   "All R y %v #i.
766   ReceiverPaid(R, y, %v) @ #i
```

```
767 ==> Ex S #j. InvoiceCreated(R, S, y, %v) @ #j & #j < #i"
```

Lemma 24 (Fees_Charged_On_Path_Possible (E)). *Non-vacuity witness. There exists a trace in which two distinct intermediaries each charge a fee on a payment with the same hash, showing that the value/fee lemmas apply to a genuine multi-hop path (and not only to a single hop).*

```
769 lemma Fees_Charged_On_Path_Possible:
770   exists-trace
771   "Ex F1 F2 y %f1 %f2 #i #j.
772   FeeCharged(F1, y, %f1) @ #i
773   & FeeCharged(F2, y, %f2) @ #j
774   & not (F1 = F2)"
```

F. Refund and Griefing

Lemma 25 (Refund_Requires_Timeout (V)). *Every refund of an HTLC is preceded by the timeout of its channel: an HTLC is refunded only after that channel has timed out.*

```
714 lemma Refund_Requires_Timeout [reuse]:
715   "All ptr y #i.
716   Refunded(ptr, y) @ #i
717   ==> Ex #j. TimedOut(ptr) @ #j & #j < #i"
```

Lemma 26 (Loss_Requires_Inaction (V)). *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then its incoming HTLC is never redeemed afterwards. An intermediary therefore loses funds only through its own inaction — failing to redeem its incoming HTLC before it times out — and never because the same HTLC was both redeemed and refunded (the two outcomes are exclusive).*

```
706 lemma Loss_Requires_Inaction:
707   "All P ptrIn ptrOut y #i #j #k.
708   ForwardHTLC(P, ptrIn, ptrOut, y) @ #i
709   & Redeemed(ptrOut, y) @ #j
710   & Refunded(ptrIn, y) @ #k
711   ==> not (Ex #l. Redeemed(ptrIn, y) @ #l)"
```

Lemma 27 (Honest_Intermediary_Refunds (E)). *Reachability witness. There exists a trace in which an honest intermediary (no key compromise) forwards an HTLC and its outgoing HTLC is subsequently refunded on timeout, with no settlement anywhere: an honest intermediary's capital can be locked up and merely returned without a payment, which is exactly the grieving denial-of-service scenario. The funds remain refundable, so safety still holds.*

```
842 lemma Honest_Intermediary_Refunds:
843   exists-trace
844   "Ex F ptrIn ptrOut y #i #j.
845   ForwardHTLC(F, ptrIn, ptrOut, y) @ #i
846   & Refunded(ptrOut, y) @ #j
847   & not (Ex a b rr p x #k. SenderSettled(a, b, rr, p, x) @ #k)
848   & not (Ex P #l. LtkCompromised(P) @ #l)"
```

G. Wormhole Fee Theft

Lemma 28 (Wormhole_Fee_Theft_Reachable (E)). *Reachability witness (attack). There exists a complete three-hop payment in which the payment settles and the receiver is paid, yet the middle hop's HTLC is never redeemed and neither intermediary ever earns a fee, with no key compromise. This is the wormhole: an honest intermediary forwards value but*

Claude
Strictness holds because Tamarin's natural numbers have no zero; say so.

Claude
Strictness holds because Tamarin's natural numbers have no zero; say so.

Claude
The lemma only gives `Ex %d. %vS = %v %+ %d`; 'plus a total fee' needs `Fee_Conservation_Hop2`.

collects nothing, because the preimage is relayed around it by two colluding endpoints.

```

784 lemma Wormhole_Fee_Theft_Reachable:
785   exists-trace
786   "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R x %v %vS %vMid
787   #t1 #t2 #t3 #t4 #t5 #t6.
788   InvoiceCreated(R, S, h(x), %v) @ #t1
789   & HTLCOffered(S, F1, ptrSF1, h(x), %vS) @ #t2
790   & HTLCForwarded1(F1, F2, S, ptrF1F2, h(x), %vS, %vMid) @ #
791   t3
792   & HTLCForwarded2(F2, R, F1, ptrF2R, h(x), %vMid, %v) @ #t4
793   & HTLCFulfilled(R, F2, ptrF2R, h(x), %v) @ #t5
794   & SenderSettled(S, F1, R, ptrSF1, x) @ #t6
795   & #t1 < #t2 & #t2 < #t3 & #t3 < #t4 & #t4 < #t5 & #t5 < #
796   t6
797   & not (Ex #i. Redeemed(ptrF1F2, h(x)) @ #i)
798   & not (Ex p1 p2 yy %f #j. FeeEarned(F1, p1, p2, yy, %f) @
799   #j)
800   & not (Ex p1 p2 yy %f #k. FeeEarned(F2, p1, p2, yy, %f) @
801   #k)
802   & not (Ex P #l. LtkCompromised(P) @ #l)"

```

Suggested rewrite of the last sentence (Lemma 28): This is the wormhole: an intermediary locks and forwards value but earns no fee, because the preimage bypasses it. In the model no party is corrupted; the Dolev–Yao adversary injects the unauthenticated fulfil on the sender’s channel, and thereby stands in for two colluding non-adjacent parties.

Lemma 29 (Wormhole_Steals_Exactly_The_Fees ($\exists$)). *The same reachable wormhole, quantified. The sender locks exactly the receiver’s amount plus the sum of the two path fees, both fees are charged, and yet neither intermediary ever earns them: the value stolen from the honest intermediary is exactly the sum of the path fees — not merely some positive loss.*

```

801 lemma Wormhole_Steals_Exactly_The_Fees:
802   exists-trace
803   "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R x %v %vS %vMid %f1 %f2
804   #t1 #t2 #t3 #t4 #t5 #t6 #c1 #c2.
805   InvoiceCreated(R, S, h(x), %v) @ #t1
806   & HTLCOffered(S, F1, ptrSF1, h(x), %vS) @ #t2
807   & HTLCForwarded1(F1, F2, S, ptrF1F2, h(x), %vS, %vMid) @ #
808   t3
809   & HTLCForwarded2(F2, R, F1, ptrF2R, h(x), %vMid, %v) @ #t4
810   & HTLCFulfilled(R, F2, ptrF2R, h(x), %v) @ #t5
811   & SenderSettled(S, F1, R, ptrSF1, x) @ #t6
812   & FeeCharged(F1, h(x), %f1) @ #c1
813   & FeeCharged(F2, h(x), %f2) @ #c2
814   & %vS = %v %+ %f1 %+ %f2
815   & #t2 < #t3 & #t3 < #t4 & #t4 < #t5 & #t5 < #t6
816   & not (Ex p1 p2 yy %f #i. FeeEarned(F1, p1, p2, yy, %f) @
817   #i)
818   & not (Ex p1 p2 yy %f #j. FeeEarned(F2, p1, p2, yy, %f) @
819   #j)
820   & not (Ex P #k. LtkCompromised(P) @ #k)"

```

H. Intermediary Safety

Lemma 30 (Intermediary_Never_Loses_Under_Liveness ($\forall$)). *If an intermediary’s outgoing HTLC is redeemed while its incoming HTLC is refunded, then its long-term key was compromised. Equivalently, an honest (non-compromised) intermediary never loses funds: whenever its outgoing HTLC is redeemed, under the liveness restrictions it also redeems its incoming HTLC.*

```

721 lemma Intermediary_Never_Loses_Under_Liveness:
722   "All P ptrIn ptrOut y #f #i #j.
723   ForwardHTLC(P, ptrIn, ptrOut, y) @ #f
724   & Redeemed(ptrOut, y) @ #i

```

```

725   & Refunded(ptrIn, y) @ #j
726   ==> Ex #r. LtkCompromised(P) @ #r"

```

Lemma 31 (Timeout_Race_Blocked ($\forall$)). *There is no trace in which an intermediary’s incoming HTLC times out strictly before its outgoing HTLC is redeemed. The early-timeout race — an incoming refund happening while the outgoing HTLC can still be settled — is blocked by the timing restrictions (in particular T2b), in the honest model.*

```

740 lemma Timeout_Race_Blocked:
741   "not (Ex P ptrIn ptrOut y #l #i #j #k.
742   ForwardHTLC(P, ptrIn, ptrOut, y) @ #l
743   & TimedOut(ptrIn) @ #k
744   & Refunded(ptrIn, y) @ #j
745   & Redeemed(ptrOut, y) @ #i
746   & #k < #i)"

```

Claude

In the model no party is corrupted (no LtkCompromised); the DY adversary injects the unauthenticated fulfil. ‘Two colluding endpoints’ is the real-world reading; say ‘stands in for’.

Claude

‘Equivalently’ is too strong: it is the contrapositive under the liveness restrictions.